

Codex & Claude Resets 追踪器

把六个官方账号的“额度重置”公告钉在一张页面上 —— 一个教你信任链、证据状态机与 Git 即数据库的迷你全栈项目

作者 Shixi Lin (@11Shixi) • github.com/shixi-11/codex-claude-resets • source-available 非商业许可 • Node 24 零依赖

一句话：它每 30 分钟去 X 上“听” Codex 和 Claude 官方的额度重置公告，**截断的一律不算数**，确认过的下次重置时间换成**你的本地时间**倒计时，九种语言静态页发布 —— 而整个“数据库”就是一个 Git 仓库。

项目速览：它解决什么问题

痛点场景（讲故事）

你正在 Codex 里写代码，突然 **limit reached**。官方说“会重置”，但消息散在 @thsottiaux、@OpenAIDevs、@OpenAI、@ClaudeDevs、@AnthropicAI、@claudeai 六个账号里：有预告、有完成通报、有重置卡补偿、有纯新闻，还夹着网友的玩笑和截图传闻。时间全写 PST/PDT，你是 UTC+8 得心算。以前的做法是：盯六个时间线 + 祈祷 + 心算时区。这个项目把这件事变成一个**确定性流水线**：**发现** → **验证** → **分类** → **展示**，人只管看结果。



图 0：整条流水线 —— 公告从 X 时间线到静态页的四跳

疑问即目录：12 个问题拆到底

这是什么？一句话说清

Q1

一个**单一用途的公告聚合追踪站**：只回答一个问题——“Codex / Claude 下次额度重置是什么时候（按我的时区）？”它是作者说的“总在脑子里碍事的小问题”的工程化解法：问题够小，所以整个项目 **src+scripts 合计约 1200 行 JS**，一个下午能读完。给工科毕业生的启示：**好工具未必大，关键是把一件事做到有据可查**。

为什么"没新消息"和"检查失败"必须分开?

Q2

追踪器最常见的谎言是把"我没能看"说成"没发生"。本项目明确：**扫描成功但零命中 = 健康；来源读不到 = 覆盖不完整**，两种情况在 health.json 里是不同的记录，页面上"上次成功检查时间"只在前者前进。同时数据超过 3 小时没刷新，浏览器会自行把状态"变陈"——**部署失败不能留下永久的绿灯**。这与知识库里的 [[健康探测三态]] (missing/broken/timeout 分态处治) 是同一条纪律：探测结果必须分状态，不能压成一个布尔值。

没有 X API (付费) 怎么拿到推文? ——双通道交叉印证

Q3

核心手法：**公共中继负责"发现"，X 官方 embed 负责"验证"，两者必须互相咬合**。FxEmbed (就是本任务取推文全文用的那个 api.fxtwitter.com) 拉取 6 个账号最近 40 条推文做候选；然后独立请求 X 官方 oEmbed，比对作者、帖子 ID、时间戳和可见文本前缀——全对得上才接受，长帖扩展文本标记为 **x-oembed+fxembed** 溯源。第三方社区站 (codexreset.org 等) **只许当"链接索引"，永远不许当证据**。分类只信验证过的原文，翻译永远排在这之后。

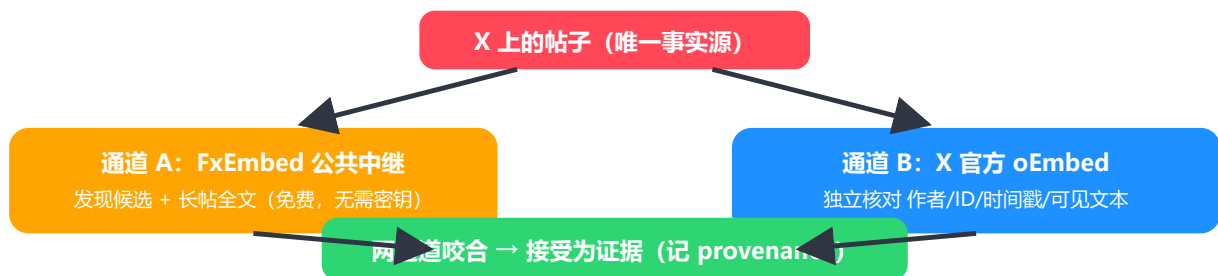


图 1: 单通道不可信, 双通道交叉印证才入账 (社区站只给虚线级的"线索"地位)

证据状态机：一条公告凭什么分四种说法?

Q4

这是全项目最有含金量的设计。每条记录打两个标签：**kind (事件种类) × state (说法状态)**。kind = global (全局重置) / banked (重置卡) / usage (额度变化) / other; state = **announced (预告将做) / reported (作者自称已做) / unconfirmed (证据不全) / information (纯资讯)**。分类器是 **src/evidence.mjs** 里 100% 确定性的正则规则 (RULES_VERSION 1.4.2)，不用 LLM 猜语义。**"已宣布完成"描述的是对方的说法，不是你的账号余额**——所以页面永远提醒你以 /status 为准。这正是 [[事实与判断分离]] 的变体：状态判定是确定性代码的事，绝不交给模糊语义。

同一句"reset", 说法状态完全不同 —— 不许混装



state 决定它能不能上倒计时、能不能算"已完成"; kind 决定它属于哪张卡片

图 2: 证据状态机四态 (实测: 预→announced, "Reset has propagated"→reported, 截断→unconfirmed)

为什么“截断”就一票否决？——保守原则

Q5

X 长帖经中继拿到的全文，只有和官方可见摘录逐字咬合才采信。只要帖子仍是截断状态，**哪怕可见部分听起来再像重置公告，也永远标 unconfirmed**。配套三条铁律：① 时间过去了不等于完成了——过了预告时刻也不会自动生成“已完成”事件；② 没有验证过的具体时刻就**绝不发明倒计时**；③ 规则升级后每轮都对全部存档原文重新分类，规则修复会自动清洗历史误收，但**网络失败和证据不足永远不能删掉已验证的完整记录**。宁可页面上写着“-- 等待确认”，也不给用户一个编造的期望。这是 [[AI 模式库与信号非证据]] 的工程化正面示范：信号只是线索，证据才可行动。

没有数据库，78 条记录存在哪？——Git 即数据库

Q6

全部状态就是 `data/` 下五个 JSON：`events.json`（78 条事件）、`platforms.json`（下一张卡片数据）、`health.json`（健康）、`translations.json`（翻译缓存）、`topic-heat.json`。GitHub Actions 每半小时跑采集器，把变更的 JSON **git commit 进公开仓库**，Vercel 的 Git 集成监听提交自动构建发布 dist/。**每次提交 = 一次数据入库 = 一次上线**，git 历史天然就是审计日志（规则修正全留痕）。零数据库、零服务器、零付费 API，公共仓库 runner 还不花私有 Actions 分钟数。与 [[快照契约与不可变只读]]、[[发布交换与健康门回滚]] 一脉相承：数据即快照，提交即发布。



失败时保留旧记录 + 写入降级健康状态 —— 数据不会被坏采集清空

图 3：Actions 提交数据，Vercel 构建发布 —— “提交即部署”

“每 30 分钟一次”的调度怎么设计？

Q7

cron 写成 **每小时 7/17/27/37/47/57 分**——特意避开 0 分整点：整点是全 GitHub 定时任务的拥堵时刻，官方文档明说 scheduled workflow 会延迟。附带两道保险：① 采集器内部对 6 个账号各自独立记账，**热闹账号 (@OpenAI 每天发模型新闻) 不能挤占冷清账号 (@ClaudeDevs) 的发现预算**；② 失败保留旧记录并发布降级健康态，绝不静默清空。手工补跑走 `workflow_dispatch`。实验实测本机直跑 `collect.mjs` 全链路成功（见实验步骤 4）。

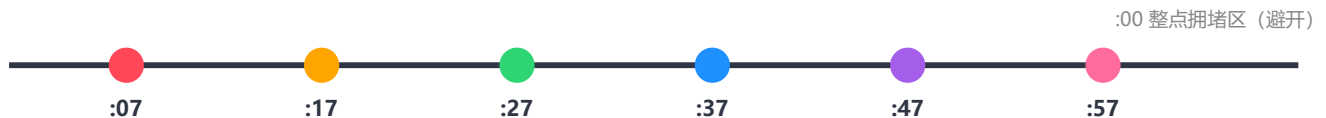


图 4：错峰调度——离整点 7 分钟起，每 10 分钟一班

Q8

平台健康怎么算？为什么 Codex 和 Claude 分开记？

health.json 里两个平台各自独立记账：**某平台"全部配置的时间线都读到 + 候选证据核验零失败"**，该平台的 **lastSuccessAt 才前进**；两者都健康整体才 healthy；任一平台超 3 小时无成功即 stale。妙处：Claude 半个月没动静（真实数据里 Claude 上次重置公告已是 31 天前）不影响 Codex 的绿灯——**"没消息"是正常输出，不是故障**。这正是 [[健康探测三态]] 里"把'查无'与'查不了'分开"的落地版。

实测字段（本机采集一轮）	值	含义
mode	profile-relay	当前采集模式（另有 x-api / community-discovery 两档降级）
sources	6×FxEmbed + X official embed 全 ok	逐源记账，候选数 19/11/16/14/21/18
status	ok	整体健康（degraded / stale 为异常）

Q9

九种语言怎么实现？为什么翻译要绑定内容哈希？

构建期把同一套模板渲染成 **9 个 locale 目录 (en/zh-Hans/zh-Hant/ja/ko/es/fr/de/ar)** 共 711 页，纯静态、无运行时翻译服务。翻译结果缓存在 **translations.json**，且绑定**"原文全文 + contentHash"**双键：原文变了一个字，哈希失配，旧译文立即作废，页面回退显示英文原文并明确标注"待翻译"。**译文永远不参与证据判定和倒计时**——翻译只是展示层，判定只看验证过的原文。这和 [[缓存]] 里的"按内容寻址失效"是同一招（参照 [[内容寻址与镜像分层]]）。

Q10

时区怎么做到"按你的本地时间显示"却不撒谎？

分工明确：**存 UTC，显示在浏览器端转访客本地时区**并标出 GMT 偏移，夏令时按日期自动处理；切换语言不改变时区。最较真的地方：公告原文写 PST 就存 UTC-08:00，写 PDT 就存 UTC-07:00，**绝不静默换算**——因为原文用哪个缩写本身就是信息。模糊的"今晚"（approximate）就保持模糊显示"约"，绝不定格成一个假精确的倒计时。所有"过去时间"（elapsed）单独一类展示，不与未来倒计时混淆。

Q11

为什么是"source-available 非商业"而不是 MIT？

仓库公开、可克隆、可本地跑、可学习修改，但**商用需要作者书面许可**（LICENSE 为自定义非商业条款）。这对学习者的实际影响：跟着做实验、改着玩、写文章完全没问题；想拿去开收费产品要发邮件谈。顺带提醒一个容易被忽略的点：推文和第三方商标的版权不属于作者，所以站点**只引用短摘录 + 原文哈希，不整篇转载**——连"引用多少"都有设计。

我能搬走哪些可复用的模式？

- ① 双通道交叉印证（发现与验证分离）
- ② 证据状态机（announced/reported/unconfirmed/information）
- ③ Git 即数据库 + 提交即部署
- ④ 错峰 cron + 分账号发现预算
- ⑤ 分平台健康记账（没消息≠故障）
- ⑥ 内容哈希绑定翻译缓存
- ⑦ 零依赖静态站（Node 自带 test/http 即够）

任何"盯公告/盯状态/盯价格"的需求（库存、版本发布、优惠券、比赛比分）都能套用这条流水线，成本为零。

这个项目给你什么能力

对你（有 Python/JS 基础的工科毕业生）的 5 个具体用途：

#	用途	怎么用
1	直接当用户	打开 shixilin.com/ai/codex-claude-resets/ 或订阅 RSS / 拉 events.json 写自己的提醒脚本
2	克隆改造成自己的追踪器	换 6 个监控账号和分类正则，就能盯任何公开公告源（比如模型发布、库存补货）
3	学零依赖 Node 工程	不用 npm install: node:test 做测试、node:http 起服务、build 脚本手写静态生成
4	学"证据链"设计	把 provenance / evidenceState / rulesVersion / contentHash 这套字段设计抄进你自己的采集项目
5	学 Actions+Vercel 免费部署	公共仓库 runner 不耗私有分钟数，整套"定时采集→提交→部署"可以白嫖

动手实验：五步真机实测（2026-10-03 全部真实执行）

环境：Windows 11 + Node v24.21.0（README 明说 Node 24 即够，**零依赖：正常构建无需 pnpm install**）。
克隆：`git clone https://github.com/shixi-11/codex-claude-resets.git` repo @ c2a9a70。

步骤 1 • 跑官方测试套件

目的：确认零依赖环境下项目逻辑完整（操作：`cd repo && node --test`）

```
i tests 60    i pass 60    i fail 0    i duration_ms 823.003
```

✅ 预期结果：60/60 全过，823ms。测试里能看到设计的影子——"relay 扩展必须匹配官方身份/时间戳/摘要"、"规则变更后历史公告仍正确分类"。

步骤 2 • 静态构建

目的：亲手验证"数据在构建期注入"（操作：`node scripts/build.mjs`）

Built 9 locales, 78 events and 711 pages.

✅ 预期结果：dist/ 出现 en/zh/ja/ko... 9 个语言目录 + events.json + feed.xml + health.json，共 711 页。

步骤 3 • 本地起服务

目的：本地完整跑起这个站（操作：`node scripts/serve.mjs`，另开终端 curl）

```
curl -s http://127.0.0.1:4187/      → HTTP 200 (313B, meta-refresh 跳 /en/)
curl -s http://127.0.0.1:4187/en/  → HTTP 200, 465099 字节完整页
curl -s http://127.0.0.1:4187/zh/ | grep title → <title>Codex与Claude重置</title>
```

✅ 预期结果：浏览器打开 `http://127.0.0.1:4187/` 能看到与线上一样的页面。坑：根路径只是跳板，真页面在 locale 目录下。

步骤 4 • 真实采集一轮（本机复现生产链路）

目的：无 token 无付费，验证采集→验证→分类→落盘全链路（操作：`node scripts/collect.mjs`）

```
{"status":"ok","records":78,"checked":6,"lastSuccessAt":"2026-10-02T21:13:46.788Z"}
health.json 实测：mode=profile-relay；6xFxEmbed 时间线全 ok
候选数：@thsottiaux 19 / @OpenAIDevs 11 / @OpenAI 16 / @ClaudeDevs 14 / @AnthropicAI 21 / @claudeai 18
```

✅ 预期结果：status ok，78 条记录保留；health.json 里逐源记账（ok/候选数）一目了然。

步骤 5 • 直调分类器，改参数看变化

目的：亲手验证证据状态机（操作：`node -e`

```
"import('./src/evidence.mjs').then(m=>console.log(m.classify(text,
{truncated:false,author:'thsottiaux'})))")
```

输入文本	完整版 →	截断版 →
Global reset landing tomorrow 10am PST...	global/announced	signal/unconfirmed
Reset has propagated. All reset for everyone.	global/reported	signal/unconfirmed
We will grant affected subscribers a reset credit...	banked/announced	signal/unconfirmed
GPT-6.1 Sol is now the default model in Codex.	other/information	other/information

✅ 预期结果：同一文本一截断就跌入 unconfirmed；无关新闻带 usage 也被拒收。RULES_VERSION=1.4.2。坑：classify 第一参数是纯字符串，传对象会报 text.replace is not a function。

一页记住

这个项目真正教你的三件事：

- ① **把"说法"和"事实"分开建模**——announced/reported/unconfirmed/information 四态让每条公告自带可信度，UI 不许撒谎。
- ② **发现与验证分离**——免费中继找线索，官方通道验真身，第三方页面只配当索引。
- ③ **最便宜的架构往往最稳**——确定性正则不用 LLM、JSON 进 Git 不用数据库、静态页不用后端，整条链路零付费、全审计、可复现（本机 5 步实验即证明）。

关联知识库：[[事实与判断分离]] · [[健康探测三态]] · [[AI味模式库与信号非证据]] · [[快照契约与不可变只读]] · [[发布交换与健康门回滚]] · [[数据源降级链]] · [[确定性脚本]] · [[缓存]] · [[内容寻址与镜像分层]]